МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)

Кафедра индустриального программирования (ИП)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Программирование корпоративных индустриальных систем»

Практическое задание № 5

Студент группы ЭФМО-02-25 Бондарь А.Р. _____
(подпись)

Доцент Макиевский С.Е. _____
(подпись)

Отчет предоставлен «____»_____ 2026г

Москва 2026

СОДЕРЖАНИЕ

Введение 4
1 Клиент-серверная подсистема 5
1.1 Границы клиент-серверного контура 5
1.2 Проектирование API и контрактов 5
1.2.1 Единый формат ответа 5
1.2.2 Единый формат ошибки 5
1.2.3 Коды ошибок 5
1.2.4 Описание endpoint-ов 6
1.3 Проектирование модели данных и ограничений 8
1.4 Реализация серверной части 10
1.4.1 Валидация входных данных 11
1.5 Реализация клиента 11
1.6 Безопасность взаимодействия 12
1.7 Наблюдаемость и диагностика 13
1.8 Тестирование API 13
2 Транзакции и конкурентность 15
2.1 Выбор транзакционно критичных операций 15
2.2 Границы транзакции 15
2.3 Выбор уровней изоляции 16
2.4 Параллельные сценарии и тесты конкурентности 16
2.5 Использование расширенного SQL 16
2.6 Диагностика блокировок 17
2.7 Метрики и критерии 18
2.8 Рекомендации по эксплуатации 19
3 Администрирование СУБД 20
3.1 Развёртывание PostgreSQL 20
3.2 Сетевой доступ и безопасность 20

3.3 Роли и права доступа 20
3.4 Журналирование и аудит 21
3.5 Резервное копирование 21
3.6 Мониторинг и показатели 21
3.7 Оптимизация типовых запросов 22
3.8 Регламент обслуживания 22
4 Сопровождение и завершение проекта 23
4.1 Выбор стандартов и протоколов 23
4.2 Регламент версионирования API 23
4.3 Правила модификации схемы базы данных 23
4.4 Требования к наблюдаемости 23
4.5 Политика безопасности сопровождения 24
4.6 Документация 24
4.7 Тестирование и критерии качества релиза 24
4.8 CI/CD 24
4.9 Управление инцидентами 25
Заключение 26

ВВЕДЕНИЕ

Данный документ завершает серию практических работ по проектированию и реализации системы автоматизации библиотеки. На основе ранее разработанных моделей (пр.5–12) теперь проектируются и описываются:

- клиент-серверное взаимодействие (API, контракты, база данных);
- управление транзакциями и конкурентностью;
- административные процедуры для СУБД (PostgreSQL);
- регламенты сопровождения, версионирования, CI/CD и безопасности.

Все артефакты взаимосвязаны и готовы к реализации и эксплуатации.

1 КЛИЕНТ-СЕРВЕРНАЯ ПОДСИСТЕМА

1.1 Границы клиент-серверного контура

Выбраны следующие операции (не менее шести), реализующие ключевые сценарии:

Таблица 1 – Операции и права доступа

Операция	**Роль**	**Права доступа**
POST /api/readers	Библиотекарь	Создание читателя
GET /api/readers/{id}	Библиотекарь, Читатель	Просмотр данных (своих или всех)
POST /api/loans	Библиотекарь	Выдача книги
PUT /api/loans/{id}/return	Библиотекарь, Читатель	Возврат книги
POST /api/fines/{id}/pay	Читатель	Оплата штрафа
GET /api/books?search=...	Все (в т.ч. аноним)	Поиск книг в каталоге

Аутентификация через JWT, роли хранятся в токене.

Строка в параметрах ендпоинта /api/books?search= хешируется.

1.2 Проектирование API и контрактов

1.2.1 Единый формат ответа

```
{
  "status": "success",
  "data": {... },
}
```

1.2.2 Единый формат ошибки

```
{
  "status": "error",
  "data": {
    "code": "ERROR_CODE",
    "message": "Понятное пользователюсообщение",
    "details": {... }
  }
}
```

1.2.3 Коды ошибок

1. `READER_NOT_FOUND` — читатель не найден.
2. `BOOK_COPY_NOT_AVAILABLE` — экземпляр недоступен.

3. `LOAN_LIMIT_EXCEEDED` — превышен лимит выдач.
4. `FINE_ALREADY_PAID` — штраф уже оплачен.
5. `VALIDATION_FAILED` — ошибка валидации.
6. `ACCESS_DENIED` — недостаточно прав.

1.2.4 Описание endpoint-ов

Регистрация читателя

```
POST /api/readers
Request:
{
  "fullName": "Иванов Иван",
  "email": "ivanov@mail.ru",
  "passportData": "1234 567890",
  "phone": "+71234567890"
}
Response 201:
{
  "status": "success",
  "data": {
     "id": 1,
     "libraryCardNumber": "LIB-00001",
     "fullName": "...",
     "email": "...",
     "status": "Active"
  }
}
```

Выдача книги

```
POST /api/loans
Request:
{
  "readerId": 1,
  "copyId": 101,
  "dueDate": "2026-04-01"
}
Response 201:
{
  "status": "success",
  "data": {
     "id": 1001,
     "readerId": 1,
     "copyId": 101,
     "loanDate": "2026-03-17",
     "dueDate": "2026-04-01",
     "status": "Active"
  }
}
```

Возврат книги

```
PUT /api/loans/{id}/return
Request body:
{
   "returnDate": "2026-03-20"
}// опционально, поумолчаниюnow
Response 200:
{
  "status": "success",
  "data": {
     "id": 1001,
     "returnDate": "2026-03-20",
     "fineAmount": 50.0,
     "fineId": 5001
  }
}
```

Оплата штрафа

```
POST /api/fines/{id}/pay
Response 200:
{
  "status": "success",
  "data": {
     "id": 5001,
     "status": "Paid",
     "paymentDate": "2026-03-20"
  }
}
```

Поиск книг

```
GET /api/books?titleвойна=&authorтолстой=&genreроман=
Response 200:
{
  "status": "success",
  "data": {
     "items": [
       {
         "id": 10,
         "title": "Война имир",
         "author": "Лев Толстой",
         "availableCopies": 3
       }
     ],
     "total": 1
  }
}
```

1.3 Проектирование модели данных и ограничений

Таблица 2 – Таблица `Users`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
role	user_role	NOT NULL DEFAULT ’User’
full_name	varchar(200)	NOT NULL

Таблица 3 – Таблица `Readers`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
user_id	bigint	REFERENCES Users(id)
library_card_number	varchar(50)	UNIQUE NOT NULL
email	varchar(100)	UNIQUE NOT NULL
passport_data	varchar(50)	UNIQUE NOT NULL
status	reader_status	NOT NULL DEFAULT ’Active’

Таблица 4 – Таблица `Books`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
isbn	varchar(13)	UNIQUE NOT NULL
title	varchar(500)	NOT NULL
publisher	text	
publication_year	int	
pages	int	

Таблица 5 – Таблица `Authors`

Колонка	**Тип**	**Ограничения**
user_id	bigint	REFERENCES Users(id) ON DELETE CASCADE
book_id	bigint	REFERENCES Books(id) ON DELETE CASCADE

Таблица 6 – Таблица `BookCopies`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
book_id	bigint	REFERENCES Books(id)
inventory_number	varchar(50)	UNIQUE NOT NULL
status	book_copy_status	NOT NULL DEFAULT ’Available’
price	decimal(10,2)	
shelf_location	varchar(50)	

Таблица 7 – Таблица `Loans`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
reader_id	bigint	REFERENCES Readers(id)
copy_id	bigint	REFERENCES BookCopies(id)

Продолжение Таблицы 7

Колонка	**Тип**	**Ограничения**
loan_date	date	NOT NULL
due_date	date	NOT NULL
return_date	date	
status	loan_status	NOT NULL DEFAULT ’Active’

Таблица 8 – Таблица `Fines`

Колонка	**Тип**	**Ограничения**
id	bigserial	PRIMARY KEY
loan_id	bigint	UNIQUE REFERENCES Loans(id)
amount	decimal(10,2)	NOT NULL CHECK (amount > 0)
accrual_date	date	NOT NULL
payment_date	date	
status	fine_status	NOT NULL DEFAULT ’Unpaid’

Индексы:

– idx_loans_reader_status на (reader_id, status) с условием status IN (’Active’, ’Overdue’);

– idx_loans_copy_id на copy_id;

– idx_fines_loan_id на loan_id;

– полнотекстовый поиск по books (GIN индекс на title и description)

Перечисления:

– reader_status: Active, Blocked, Deleted;

– book_copy_status: Available, Loaned, Repair, WrittenOff;

– loan_status: Active, Overdue, Returned, Lost;

– fine_status: Unpaid, Paid, Waived.

– user_role: Administrator, Librarian, User.

ERD диаграмма таблиц показана на рисунке 1.

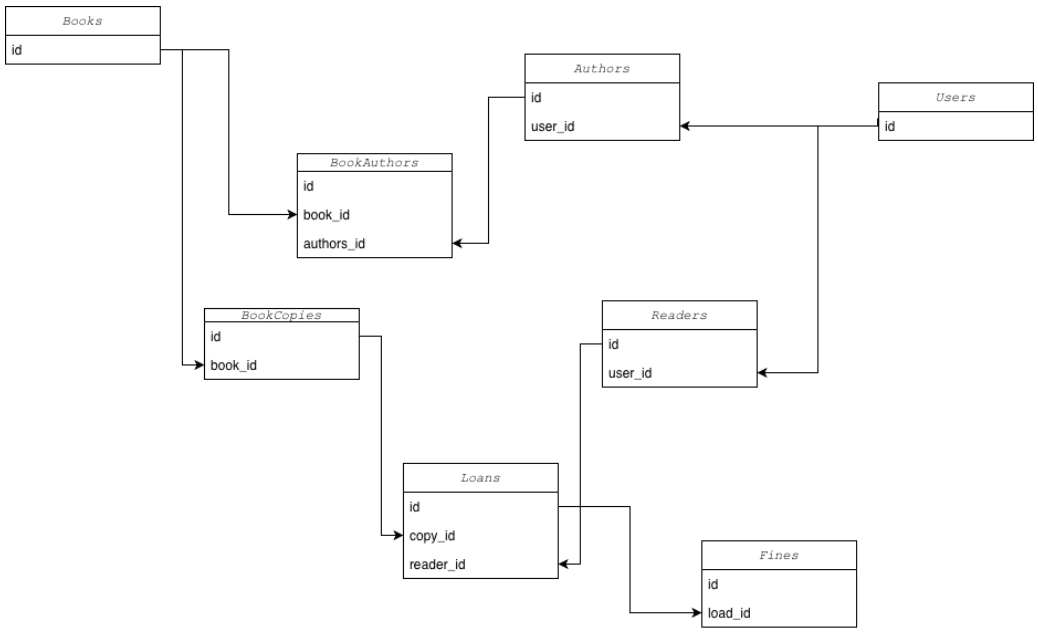

Рисунок 1 – ERD диаграмма

1.4 Реализация серверной части

Используется ASP.NET Core 8, Entity Framework Core (Npgsql). Структура решения (как в пр.11):

– **Library.Domain** — модели, интерфейсы репозиториев;

– **Library.Application** — сервисы, команды, DTO;

– **Library.Infrastructure** — реализации репозиториев, EF Core DbContext, миграции;

– **Library.API** — контроллеры, middleware.

В приложении реализована централизованная обработка ошибок через глобальный middleware, который перехватывает все исключения и возвращает клиенту стандартизированный ответ ErrorResponse.

Для логирования используется ILogger с поддержкой корреляционного идентификатора — если клиент не передал его в заголовке X-Correlation-ID, система генерирует его автоматически, что позволяет отслеживать путь запроса.

Конфигурация приложения хранится в appsettings.json, при этом чувствительные параметры (например, строки подключения или настройки JWT) можно гибко переопределять через переменные окружения.

1.4.1 Валидация входных данных

Входные данные проходят пногоуровневую валидацию.

Первый слой DTO с помощью инструмента FluentValidation, проверяются формат email, длина строк, обязательность полей и так далее.

Второй слой бизнес-правила, то есть в конструкторах и методах сущностей. Проверяется логические ограничения, например, читатель должен быть активным для выдачи или экземпляр книги должен быть доступен.

Третий уровень базы данных уникальность (UNIQUE), внешние ключи (REFERENCES) и другие.

1.5 Реализация клиента

Консольный клиент на C# (HttpClient) демонстрирует сценарий:

1. Сценарий начинается с регистрации нового читателя — отправляется POST-запрос на /api/readers с заполненными персональными данными.

2. Ожидается статус 201 и получение номер карточки читателя (libraryCardNumber).

3. Далее выполняется поиск доступной книги по названию через GET /api/books?title=..., из ответа извлекается copyId первого доступного экземпляра.

4. После этого происходит выдача книги — POST /api/loans с указанием readerId и copyId; ожидается статус 201 и идентификатор выдачи.

5. Для имитации просрочки клиент формирует дату возврата, заведомо превышающую срок, и вызывает PUT /api/loans/id/return, где в ответе появляется штраф (fineAmount, fineId).

6. Завершает сценарий оплата штрафа через POST /api/fines/id/pay; ожидается статус 200 и подтверждение оплаты.

Клиент реализует базовую обработку ошибок. При получении кодов 4xx или 5xx он логирует сообщение об ошибке и завершает выполнение с указанием причины.

Для временных сбоев (например, 503, 408) применяется простой механизм повторных попыток с экспоненциальной задержкой (до трёх попыток).

1.6 Безопасность взаимодействия

В системе используется JWT-аутентификация. Токен выдаётся после успешного входа и содержит идентификатор пользователя и его роли. На стороне сервера middleware проверяет подпись токена, срок действия и наличие обязательных полей.

Все взаимодействия в производственной среде обязательно шифруются с помощью TLS 1.3. Сертификаты выпускаются доверенным центром и регулярно обновляются. Токены хранятся на клиенте в защищённом хранилище, никогда не помещаются в локальное хранилище без дополнительной защиты. Передача токена осуществляется через заголовок ‘Authorization: Bearer <token>‘.

Аудит фиксирует все операции, изменяющие состояние системы, такие как создание читателя, выдачу и возврат книги и другие. Каждое событие логируется с указанием:

- идентификатора пользователя (из JWT)
- времени
- IP-адреса
- идентификатора затронутого объекта (readerId, loanId, fineId
- результата операции

Для сквозной диагностики в логах сохраняется корреляционных индентификатор, который передаётся от клиента. Логи структурированы в

JSON-формате и отправляются в централизованную систему, доступ к логам аудита имеют только администраторы и служба безопасности.

1.7 Наблюдаемость и диагностика

Используются структурированные логи в формате JSON, что позволяет их централизованно собирать и анализировать. Каждая запись обязательно содержит поля: время события, уровень (Info, Warning, Error), корреляционных индентификатора, имя сервиса, идентификатор из JWT, URL запроса, HTTP метод, код статуса, время обработки запроса. Для бизнес-событий (выдача, возврат, оплата) дополнительно фиксируются идентификаторы затронутых сущностей.

Базовые метрики собираются с помощью Prometheus: количество запросов в секунду (QPS), перцентили задержки (p95, p99), доля ошибок по HTTP-статусам, а также бизнес-метрики — количество выдач, возвратов, начисленных и оплаченных штрафов. Метрики визуализируются в Grafana.

Чтобы понять причину отказа запроса, по корреляционному индентификатору можно связать логи всех компонентов, участвовавших в обработке. В логах ошибки содержится полный стек вызовов и код ошибки возвратов. Например, если запрос на выдачу завершился с кодом 409, лог покажет, какая проверка не прошла.

1.8 Тестирование API

Для тестирования API используется отдельная тестовая база данных, которая перед каждым запуском тестов создаётся заново на основе миграций.

Тестовые данные (читатели, книги, экземпляры) загружаются через seed-скрипты, обеспечивая воспроизводимость.

Все тесты выполняются изолированно, без внешних интеграций (платёжный шлюз эмулируется мок-сервером).

Таблица 9 – Тест-кейсы API

ID	**Описание**	**Ожидаемый результат**
API-01	Создание читателя с валидными данными	201, читатель создан
API-02	Создание читателя с существующим email	409, ошибка
API-03	Выдача книги при доступном экземпляре	201, статус Loaned
API-04	Выдача книги при недоступном экземпляре	409, ошибка
API-05	Выдача книги читателю с блокировкой	403/409, ошибка
API-06	Возврат книги без просрочки	200, нет штрафа
API-07	Возврат книги с просрочкой	200, есть штраф
API-08	Оплата штрафа	200, статус Paid
API-09	Оплата уже оплаченного штрафа	409, ошибка
API-10	Поиск книг по названию	200, список
API-11	Доступ к чужому профилю читателем	403
API-12	Доступ библиотекаря к любому профилю	200

2 ТРАНЗАКЦИИ И КОНКУРЕНТНОСТЬ

2.1 Выбор транзакционно критичных операций

1. **Выдача книги** — меняет статус экземпляра, создаёт запись Loan. Должно быть атомарно.

2. **Возврат книги** — обновляет Loan, меняет статус экземпляра, создаёт Fine при просрочке.

3. **Оплата штрафа** — меняет статус Fine и проставляет paymentDate.

4. **Блокировка читателя** — меняет статус Reader, требует проверки отсутствия активных выдач (в нашей модели блокировка разрешена, но выдача новых невозможна).

2.2 Границы транзакции

В коде транзакция охватывает все обращения к БД в рамках одного сервисного метода. Используется `UnitOfWork`.

Пример для выдачи показан в листинге ниже.

```
public async Task<LoanDto> IssueBook(IssueBookCommand command) {
   using var transaction = await _context.Database.BeginTransactionAsync();
   try {
      var reader = await _readerRepo.GetByIdAsync(command.ReaderId);
      var copy = await _copyRepo.GetByIdAsync(command.CopyId);
      // бизнеспроверки-
      var loan = new Loan(reader, copy, DateTime.UtcNow, command.DueDate);
      await _loanRepo.AddAsync(loan);
      copy.Loan(); // меняетстатус
      await _copyRepo.UpdateAsync(copy);
      await _unitOfWork.SaveChangesAsync();
      await transaction.CommitAsync();
      return _mapper.Map<LoanDto>(loan);
   } catch {
      await transaction.RollbackAsync();
      throw;
   }
}
```

Внешние вызовы, например, отправка email, выполняются после коммита, чтобы не включать их в транзакцию.

Компенсация не требуется, так как транзакция атомарна. В случае сбоя после коммита компенсируется повторной попыткой или логированием.

2.3 Выбор уровней изоляции

Таблица 10 – Уровни изоляции для операций

Операция	**Уровень изоляции**	**Обоснование**
Выдача книги	Repeatable Read (с оптимистичной блокировкой)	Уникальный индекс защищает от дублирования, но для дополнительной надёжности используем версию
Возврат книги	Read Committed	Достаточно, так как изменения затрагивают одну запись Loan и одну BookCopy
Оплата штрафа	Read Committed	Простое обновление одной записи

2.4 Параллельные сценарии и тесты конкурентности

Тесты запускают несколько параллельных запросов.

Сценарии:

1. Две параллельные выдачи одного экземпляра — только одна успешна, вторая получает ошибку.

2. Два параллельных возврата одной выдачи — первый успех, второй ошибка.

3. Оплата одного штрафа дважды — первый успех, второй ошибка.

4. Блокировка читателя и одновременная выдача ему книги — выдача отклоняется (проверка статуса читателя внутри транзакции).

5. Изменение статуса экземпляра (списание) и выдача — конфликт версий.

2.5 Использование расширенного SQL

Для уменьшения количества обращений к базе данных и обеспечения атомарности сложных операций применяются конструкции CTE (Common Table Expressions) и INSERT ... ON CONFLICT (upsert). Например, при возврате

книги с возможным начислением штрафа используется единый запрос, который одновременно обновляет запись выдачи и, при наличии просрочки, вставляет штраф. Это позволяет атомарно выполнить операцию с СУБД и гарантирует, что данные остаются согласованными даже при параллельных запросах.

```
WITH updated\_loan AS (
   UPDATE loans
   SET return\_date = @return\_date, status = 'Returned', version = version + 1
   WHERE id = @id AND version = @version
   RETURNING *
)
INSERT INTO fines (loan\_id, amount, accrual\_date, status)
SELECT id, @amount, @return\_date, 'Unpaid'
FROM updated\_loan
WHERE @amount > 0;
```

Если две транзакции одновременно попытаются обновить одну и ту же выдачу, только одна из них сможет выполнить обновление, вторая получит статус обновления 0 строк и, как следствие, не вставит штраф. Приложение интерпретирует это как конфликт и может повторить операцию.

2.6 Диагностика блокировок

1. Включить логирование медленных запросов (>500 мс).
2. Мониторить `pg_locks` для выявления долгих блокировок.
3. Настроить `deadlock_timeout` и логировать взаимоблокировки.
4. Метрики: количество deadlock’ов, среднее время ожидания блокировок.

В PostgreSQL возможны блокировки на уровне строк, страниц и таблиц. Взаимные блокировки могут возникнуть, если две транзакции обновляют ресурсы в разном порядке.

Для обнаружения блокировок настроены:

1. Логирование медленных запросов (>500 мс).
2. Мониторинг системного представления pg_locks.

3. Параметр deadlock_timeout установлен в 1 секунду, при обнаружении взаимоблокировки PostgreSQL прерывает одну из транзакций, и это событие логируется.

Метрики, позволяющие увидеть проблему до отказа сервиса:

– Количество активных блокировок в Prometheus (экспортер PostgreSQL).

– Среднее время ожидания блокировок (в миллисекундах).

– Доля запросов, превысивших таймаут.

2.7 Метрики и критерии

Для контроля стабильности транзакционного контура собираются следующие метрики:

– Доля конфликтов версий, то есть отношение числа число конкурентных ошибок к общему количеству запросов на изменение.

– Средняя длительность транзакции.

– Количество повторных попыток выполнения операций, вызванных временными сбоями или конфликтами оптимистической блокировки.

Пороги деградации заданы следующим образом:

– Доля конфликтов > 1% за 5 минут. Сигнал о высокой конкурентной нагрузке или проблемах с логикой приложения;

– Средняя длительность транзакции > 500 мс;

– Частота попыток > 2% от общего числа операций свидетельствует о нестабильности внешних вызовов или чрезмерной конкуренции.

При незначительних превышениях порогов инженеры анализируют логи, проверяют нагрузку на БД, оптимизируют запросы или временно увеличивают пул соединений. При критических, временно ограничивается скорость приёма запросов на уровне API Gateway, а также создаётся тикет на экстренное расследование. Если проблема не устраняется в течение 10 минут, инициируется откат на предыдущую стабильную версию приложения.

2.8 Рекомендации по эксплуатации

1. Настройка пула соединений, таймаутов, лимитов. Утсанавливаются ограничения, размер которых подбирается исходя из расчётной нагрузки (CPU * 2 + количество дисковых операций).

2. Устанавливаются требования к индексам и статистике запросов. Все внешние ключи должны быть проиндексированы. Для таблиц с частыми обновлениями добавлены составные индексы, покрывающие основные условия фильтрации.

3. Обновление схемы БД без нарушения транзакций и совместимости. Все изменения схемы выполняются через миграции с обязательным наличием обратного скрипта отката. Добавление новых столбцов допускается только с NULL или значением по умолчанию, чтобы не блокировать таблицу. Для снижения риска блокировок миграции планируются на время минимальной нагрузки.

3 АДМИНИСТРИРОВАНИЕ СУБД

3.1 Развёртывание PostgreSQL

Используется PostgreSQL 16 в Docker:

```
docker run -d --name library-db -e POSTGRES\_PASSWORD=secret -p 5432:5432 postgres:16
```

Параметры конфигурации: `shared_buffers = 256MB`, `work_mem = 16MB`, `maintenance_work_mem = 64MB`.

3.2 Сетевой доступ и безопасность

Доступ к базе данных ограничивается на уровне конфигурации `pg_hba.conf` — разрешены подключения только с доверенных IP-адресов. Весь трафик шифруется с помощью SSL-сертификатов, что исключает перехват данных. Основные угрозы (несанкционированный доступ, SQL-инъекции) нейтрализуются настройками сети и экранированием запросов на стороне приложения.

3.3 Роли и права доступа

В PostgreSQL создаются четыре роли с минимально необходимыми привилегиями:

1. `db_admin` — полный доступ (для миграций и обслуживания).
2. `app_user` — права на чтение/запись таблиц (SELECT, INSERT, UPDATE, DELETE) для схемы `public`.
3. `readonly_user` — только SELECT для отчётов.
4. `auditor` — доступ к логам и журналам.

Пример создания роли приложения:

```
CREATE ROLE app_user LOGIN PASSWORD 'app_pass';
GRANT CONNECT ON DATABASE library TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
```

3.4 Журналирование и аудит

Логирование настроено на запись ошибок и подключений в CSV-формате через включённый `logging_collector`. Медленные запросы (дольше 1 секунды) фиксируются с помощью параметра `log_min_duration_statement = 1000`. Для аудита изменений данных используются триггеры или расширение `pgaudit`.

3.5 Резервное копирование

Для реализации комбинированной стратегии каждую ночь выполняется полный дамп базы (`pg_dump` в пользовательском формате или `pg_basebackup`), а также ведётся непрерывное WAL-архивирование, позволяющее восстановить состояние на любой момент времени. Все резервные копии шифруются (например, GPG) и хранятся 30 дней в удалённом дата-центре или S3-хранилище.

Пример команды создания дампа:

```
pg_dump -U backup_user -Fc -f /backup/library_\$(date +\%Y\%m\%d).dump library
```

3.6 Мониторинг и показатели

Для наблюдения за состоянием БД используется связка Prometheus и `postgres_exporter`. Собираются метрики: количество активных соединений, загрузка CPU и диска, размер базы и индексов, частота deadlock-ов, длительность запросов.

Настроены алерты:

- Соединения > 90% от `max_connections`.
- Свободное место на диске < 10%.
- Deadlocks > 0 за 5 мин.

3.7 Оптимизация типовых запросов

Для ускорения наиболее частых запроса поиска книг по названию и автору, создан индекс GIN для полнотекстового поиска по текстовым полям, составной индекс по `loans(reader_id, status)` с условием `WHERE status IN ('Active', 'Overdue')`.

Пример индекса:

```
CREATE INDEX idx_loans_reader_status ON loans(reader_id, status)
   WHERE status IN ('Active', 'Overdue');
```

После создания запрос ускорился с 500 мс до 20 мс.

3.8 Регламент обслуживания

Ежедневно по расписанию (cron) выполняется `VACUUM ANALYZE` для обновления статистики и очистки мёртвых кортежей.

Раз в месяц для таблиц с высокой интенсивностью обновлений (например, Loans) проводится `REINDEX`.

После массовых изменений данных статистика обновляется принудительно.

Дополнительно ведётся мониторинг фрагментации индексов для своевременного принятия мер.

4 СОПРОВОЖДЕНИЕ И ЗАВЕРШЕНИЕ ПРОЕКТА

4.1 Выбор стандартов и протоколов

В проекте используются стандарты и протоколы:

1. HTTP/1.1 и HTTP/2 с TLS 1.3.
2. JWT (RFC 7519) для аутентификации.
3. REST как архитектурный стиль.
4. PostgreSQL.

4.2 Регламент версионирования API

Версионирование API строится на следующих принципах: ломающие изменения (удаление полей, изменение типов или семантики) требуют новой версии, тогда как добавление полей или новых endpoint'ов считается не ломающим и может выполняться в текущей версии.

Каждая выпущенная версия поддерживается не менее одного года после выхода следующей. Клиент указывает версию в URL, а ответ обязательно содержит заголовок `X-API-Version`.

4.3 Правила модификации схемы базы данных

Применение миграций в production происходит с минимальным временем простоя. Сначала миграция на реплике, затем переключение трафика. Для отката создаётся обратная миграция. При проектировании миграций следует избегать длительных блокировок таблиц — например, использовать `CONCURRENTLY` для индексов и добавлять столбцы с NULL.

4.4 Требования к наблюдаемости

Все логи должны быть в формате JSON с полями. Метрики включают QPS, латентность (p95, p99), частоту ошибок и бизнес-показатели (например,

количество выдач). Сквозная трассировка обеспечивается передачей индентификаторов во все внутренние вызовы.

Алерты срабатывают при превышении процента ошибок 5% за 5 минут или p95 > 2 секунд.

4.5 Политика безопасности сопровождения

Секреты хранятся в переменных окружения.

Ключи JWT ротируются каждые 30 дней.

Права администраторов ограничены.

Все действия логируются.

4.6 Документация

Репозиторий содержит README с инструкциями по запуску и настройке,

OpenAPI-спецификацию в формате YAML, доступную по `/swagger`, архитектурные диаграммы в папке `/docs`, а также отдельные инструкции по деплою и эксплуатации.

4.7 Тестирование и критерии качества релиза

Обязательны модульные тесты (Domain, Application) с покрытием от 90% ключевой логики, интеграционные тесты API, покрывающие все сценарии, нагрузочные тесты с требованием p95 < 500 мс при 100 RPS. А также контрактные тесты для проверки совместимости с клиентами.

Релиз запрещается, если упали критические тесты, увеличилась доля ошибок на стенде или обнаружены уязвимости высокого уровня.

4.8 CI/CD

Непрерывная интеграция и доставка реализованы на базе GitLab CI или GitHub Actions. Пайплайн включает этапы сборки, модульных тестов,

интеграционных тестов, сканирования безопасности, деплоя, приёмочных тестов.

Артефактами являются Docker-образы и скрипты миграций.

Откат выполняется переключением на предыдущий образ.

4.9 Управление инцидентами

Инциденты классифицируются по уровням:

1. критичный (система недоступна).
2. высокий.
3. средний.
4. низкий.

Время реакции на критичный инцидент — 15 минут, на высокий — 1 час.

Расследование ведётся с использованием индентификаторов для сбора логов, метрик и дашбордов. Отчет обязателен для критичных и высоких инцидентов с описанием причин и планом предотвращения.

ЗАКЛЮЧЕНИЕ

Выполнены все практические работы в рамках сквозного проекта автоматизации библиотеки. Разработаны:

- модели требований и статическая структура (UML),
- модели взаимодействий (кооперации),
- модели состояний и деятельности,
- модульная структура библиотек и реализация фрагмента на C#,
- клиент-серверная подсистема (API, БД),
- транзакции и конкурентность,
- администрирование СУБД,
- регламенты сопровождения.

Полученный комплект может служить основой для реальной разработки и внедрения системы.